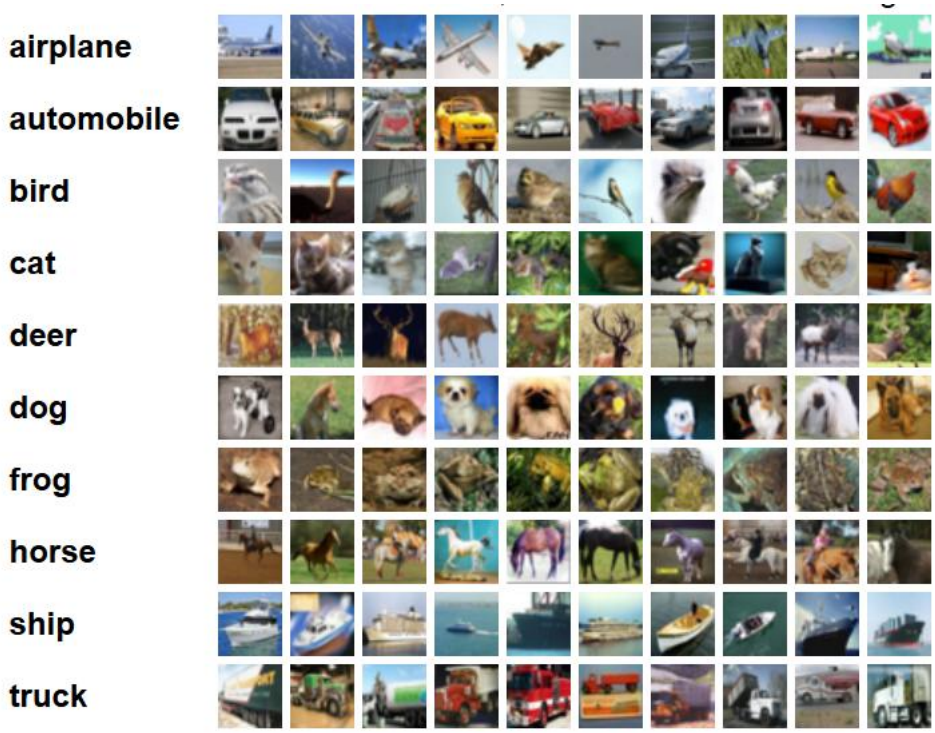
## Dataset & task

CIFAR-10 (Krizhevsky, 2009, *Learning Multiple Layers of Features from Tiny Images*, https://www.cs.toronto.edu/~kriz/cifar.html): 60,000 colour photos, 32x32 pixels, 10 balanced classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). I predict which of the 10 classes each image belongs to. Split: the official 10,000-image test set is kept untouched, and the 50,000 training images are split with the fixed seed into 40,000 train / 10,000 validation. So the final split is 40,000 train / 10,000 val / 10,000 test.



## Model & training choices

A CNN, because photos have local structure and conv layers share their weights across positions (far fewer parameters than an MLP on flattened pixels). The baseline is the starter CNN (2 x Conv-ReLU-MaxPool, then a 128-unit linear layer with dropout 0.25). My model, BetterCNN, adds a 3rd conv block (64 -> 128 -> 256 channels) and BatchNorm after every conv.

Loss: cross-entropy.

Optimizer: Adam, lr 1e-3 (3e-3 only in run 2), batch size 128, 8 epochs, weight decay 1e-4. Regularization: dropout, weight decay (L2), and data augmentation (random horizontal flip + random crop with padding 4, training set only). Seed fixed at 0.

The final model was chosen using validation accuracy only: BetterCNN (BatchNorm + 3rd conv block) with 75.94%.

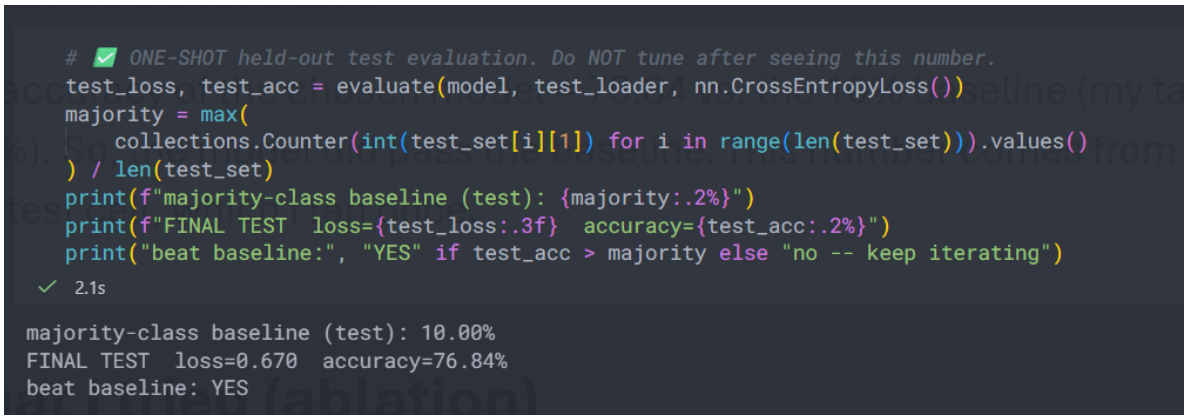
```
CNN(
  (features): Sequential(
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (head): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=4096, out_features=128, bias=True)
    (2): ReLU()
    (3): Dropout(p=0.25, inplace=False)
    (4): Linear(in_features=128, out_features=10, bias=True)
  )
)
trainable parameters: 545,098
```

## Baseline

Majority-class predictor (always guess the same class). CIFAR-10 is balanced, so it scores 10%.

## Final test metric

Test accuracy of the chosen model = 76.84% vs. the 10% baseline (my target was > 65%). So, the model did pass the baseline. This number comes from the one-shot test cell, which I ran once.



```
# ✅ ONE-SHOT held-out test evaluation. Do NOT tune after seeing this number.
test_loss, test_acc = evaluate(model, test_loader, nn.CrossEntropyLoss())
majority = max(
    collections.Counter(int(test_set[i][1]) for i in range(len(test_set))).values()
) / len(test_set)
print(f"majority-class baseline (test): {majority:.2%}")
print(f"FINAL TEST  loss={test_loss:.3f}  accuracy={test_acc:.2%}")
print("beat baseline:", "YES" if test_acc > majority else "no -- keep iterating")
```

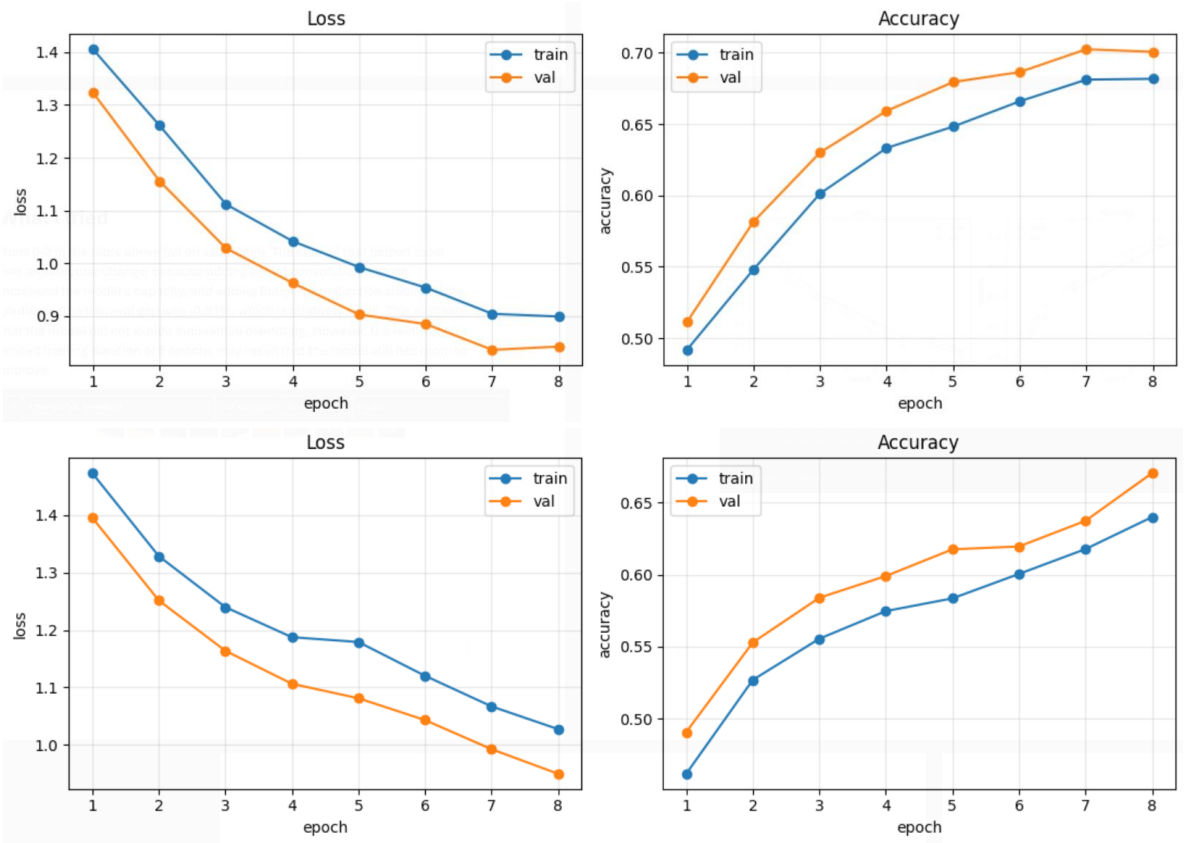
✓ 2.1s

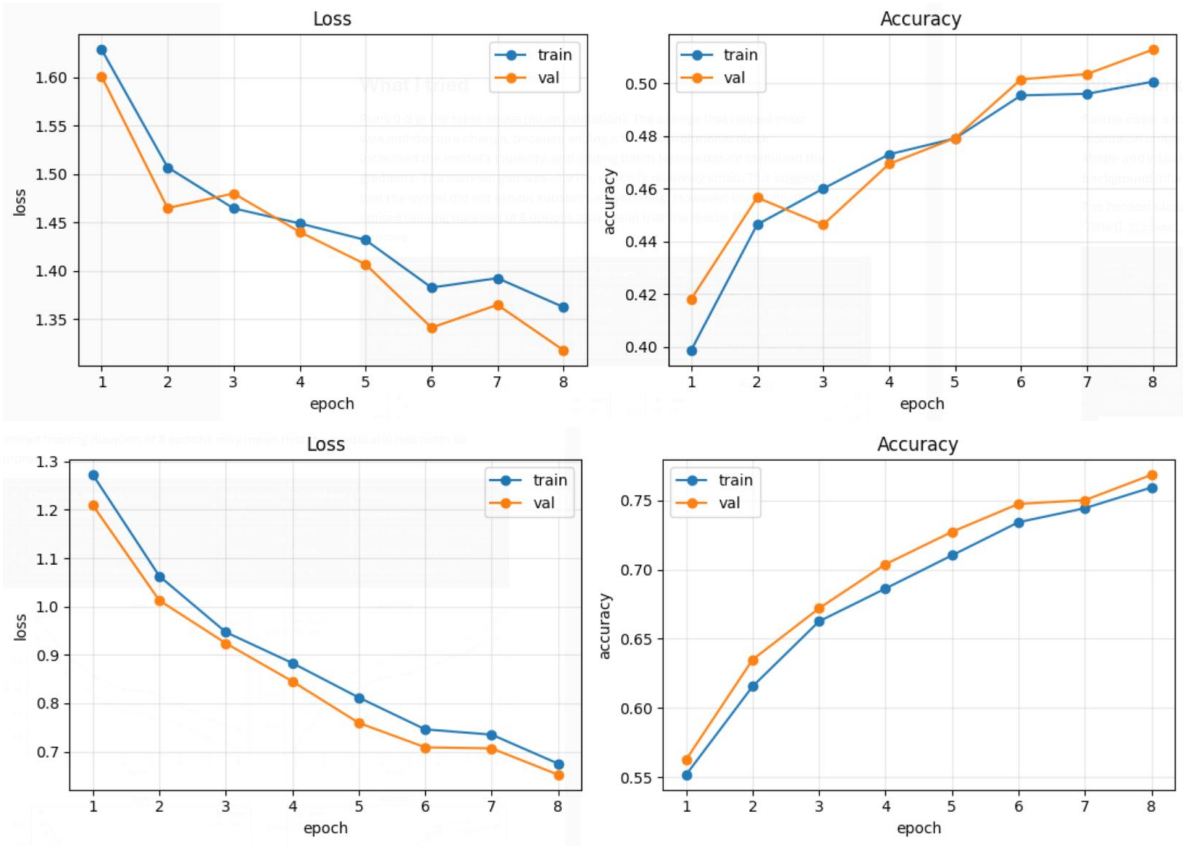
```
majority-class baseline (test): 10.00%
FINAL TEST  loss=0.670  accuracy=76.84%
beat baseline: YES
```

## What I tried

Runs 0-3 in the table above (all on validation). The change that helped most was architecture change, because adding a third convolutional block increased the model's capacity, and adding Batch Normalization stabilized the gradients. The train-val gap was -0.91%, which is relatively small. This suggests that the model did not exhibit substantial overfitting. However, the relatively limited training duration of 8 epochs may mean that the model still has room to improve.

| # | Change (vs. baseline) | Val accuracy | Train–val gap | Notes |
|---|---|---|---|---|
| 0 | baseline (starter model) | *70.07%* | *-1.89%* | *the run above* |
| 1 | +dropout 0.5 (p_drop 0.25 -> 0.5) | *67.05%* | *-3.05%* | *same CNN, only dropout changed* |
| 2 | lr 3e-3 (1e-3 -> 3e-3) | *51.28%* | *-1.22%* | *same CNN, only lr changed* |
| 3 | BetterCNN (+BatchNorm, +3rd conv block) | *75.94%* | *-0.91%* | *architecture change only* |



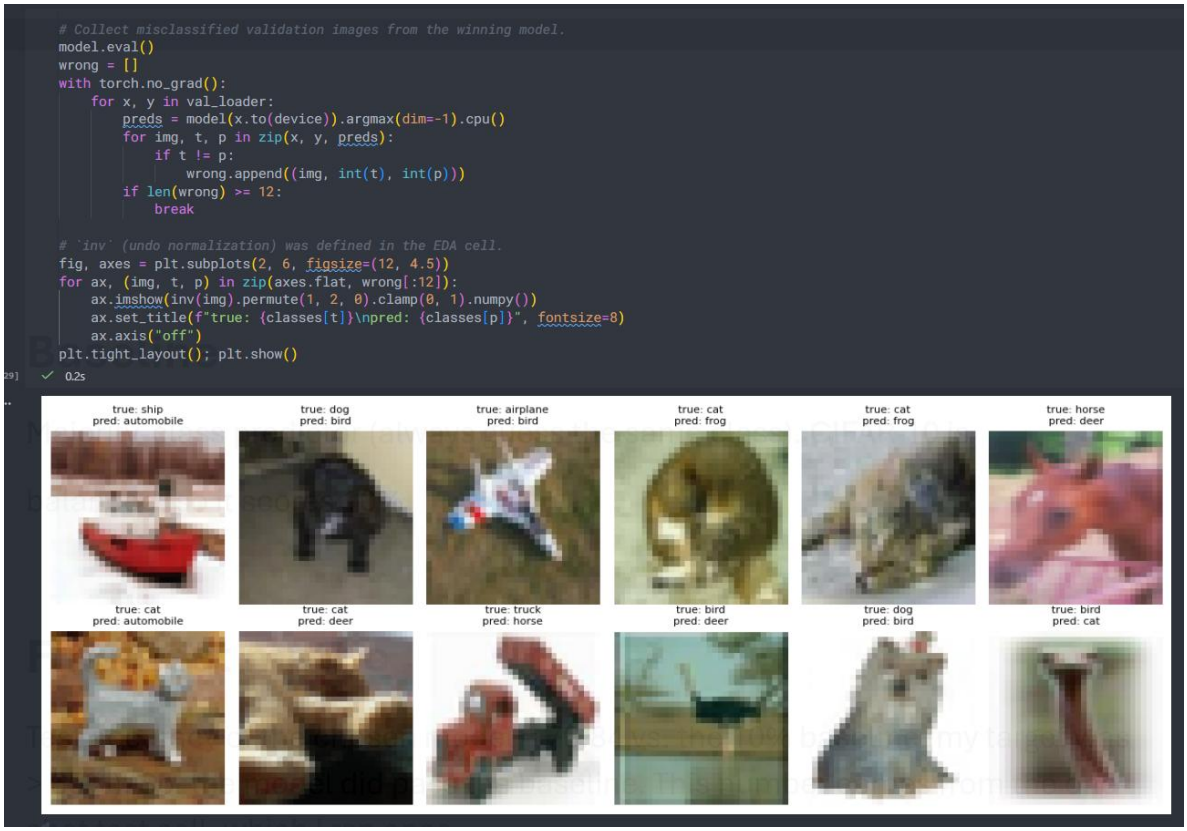



(learning curves from baseline, +dropout, lr 3e-3, BetterCNN in orders)

## What worked / what failed

Failure case: a true cat predicted as a frog, probably because the low-resolution image made it difficult for the model to distinguish the animal's shape and visual features. The model may also have relied on color or background information instead of more distinctive features.

The hardest classes on validation were [('cat', '53%'), ('bird', '64%'), ('dog', '70%')]. (‘class’, ‘val%’)




## With more time

I would try a ResNet-style network with skip connections, a learning rate schedule, stronger augmentation, and early stopping on validation loss.

My models are small and I only trained for 8 epochs, so there is probably room to improve.